HrZ-HRMS App

Complete Full-Stack Architecture & Technical Analysis Report

Flutter Mobile App | February 2026 | Version 0.1.0 | Android + iOS

Table of Contents

1. Project Overview
2. Architecture Pattern
3. State Management
4. Networking / API Layer
5. Real-Time Communication (WebSocket)
6. Storage Strategy
7. Push Notifications
8. Navigation System
9. Firebase Integration
10. Theme & UI
11. Authentication Flow
12. Feature Modules
13. Key Packages Summary
14. Architecture Diagram

1 Project Overview

HrZ_hrms is a production-grade **Human Resource Management System (HRMS)** mobile application built with Flutter. It serves as an internal employee portal for organizations, supporting real-time chat, leave management, attendance tracking, internal mail, meetings, and more.

PROPERTY	VALUE

App Name	HrZ_hrms
Package Name	hrtz_hrms
Version	0.1.0
Framework	Flutter (Dart SDK ^3.9.0)
Platforms	Android + iOS
Backend REST	AWS API Gateway (us-east-1)
Backend Socket	Render.com (Socket.IO)
Push Notifications	Firebase Cloud Messaging (FCM)
Crash Reporting	Firebase Crashlytics

2 Architecture Pattern

The project follows a **Feature-First Clean Architecture** with a clear separation of concerns across 4 top-level layers.

```
lib/
├── config/              # App-wide configuration
│   ├── navigation/      # GoRouter setup + route constants
│   └── theme/           # AppTheme (light mode)
│
├── core/                # Shared infrastructure
│   ├── api/             # HTTP layer (ApiHelper + services)
│   ├── constants/       # AppConstants (URLs, sizes, durations)
│   ├── errors/          # Custom exceptions + error screen
│   ├── models/          # 12 shared data models
│   ├── services/        # Storage, notifications, logging
│   ├── socket/          # Real-time socket layer
│   └── utils/           # Utility helpers
│
├── features/            # 12 feature modules
│   ├── auth/            # Login, OTP, password, NDA, onboarding
│   ├── chat/            # 1-on-1 & group chat
│   ├── complaint/       # Complaint management
│   ├── holiday/         # Holiday calendar
│   ├── home/            # Dashboard, attendance clock-in/out
│   ├── hr_services/     # Resignation
│   ├── leave/           # Leave requests & admin approval
│   ├── mail/            # Internal mail system
│   ├── meet/            # Meeting scheduling
│   ├── settings/        # Permissions screen
│   ├── social/          # Social hub / posts
│   └── tasks/           # Task management
```

```
│
└── widgets/            # Global reusable widgets
```

Each feature follows an internal **Provider → Service → ApiHelper** pattern:

> Screen (UI) → Provider (state) → Service (API calls) → ApiHelper (HTTP) → Backend

3 State Management

Uses **Provider** (`provider: ^6.1.5+1`) with the **ChangeNotifier** pattern throughout the app.

PROVIDER	RESPONSIBILITY	SCOPE
AppStateProvider	App readiness flag (splash → main)	Global
AuthProvider	Login, logout, session, profile, employee list	Global
SocketProvider	1-on-1 chat messages, online users, typing, unread counts	Global
GroupSocketProvider	Group chat messages and group management	Global
MeetProvider	Meeting list state	Global
LeaveProvider	Leave requests & history	Global
HolidayProvider	Holiday list	Global
TasksProvider	Task list	Global
SocialProvider	Social posts feed	Global
ComplaintProvider	Complaint list	Global
MailProvider	Inbox (auto-fetched on startup)	Global
ClockProvider	Attendance clock-in/out (auto-fetched on startup)	Global
ResignationProvider	Resignation form state	Route-scoped

4 Networking / API Layer

🌐 REST API

- Package: `http: ^1.5.0`
- Base URL: AWS API Gateway
- Region: us-east-1
- Auth: Bearer JWT token
- Methods: GET, POST, PUT, PATCH, DELETE

🔌 SOCKET SERVER

- Package: socket_io_client ^3.1.2
- Host: Render.com
- URL: hrms-socket.onrender.com
- Protocol: Socket.IO (WebSocket)
- Auto-reconnect on disconnect

ApiHelper — Generic HTTP Client

A single `ApiHelper` class wraps all HTTP calls with automatic token injection, request/response logging, and error handling:

FEATURE	DETAIL
Token injection	Auto-reads JWT from SecureStorage and adds `Authorization: Bearer` header
Response wrapper	Generic `ApiResponse<T>` with success, data, message, statusCode
Error handling	Custom exceptions: BadRequest, Unauthorized, Forbidden, NotFound, Server
Log sanitization	Redacts password, token, otp, secret from logs
Network error	Catches SocketException → returns "No Internet Connection"

Network Service Classes

SERVICE	KEY ENDPOINTS
AuthService	login, getOTP, verifyOTP, changePassword, getEmployeeList, getEmployeeInfo, updateProfile, updateFCMToken, leave CRUD, attendance
ClockService	Clock-in / Clock-out
MailService	Inbox, sent, trash, compose, reply, delete
MeetService	Get meets, schedule meet
SocialService	Get posts, create post
TaskService	Get tasks, update task
HolidayService	Get holidays
ComplaintService	Get / submit complaints
ResignationService	Submit resignation

5 Real-Time Communication (WebSocket)

A 3-layer socket architecture separates connection management, event abstraction, and UI state:

```
SocketService  (singleton)   → Raw socket.io connection + reconnect logic
      ↓
SocketManager (singleton)    → Emit/listen event abstraction
      ↓
SocketProvider               → 1-on-1 chat UI state (ChangeNotifier)
GroupSocketProvider          → Group chat UI state (ChangeNotifier)
```

📤 EMITTED EVENTS (1-ON-1)

- join
- sendMessage
- typing
- messageRead
- markConversationAsRead
- deleteMessage
- clearConversation

📥 LISTENED EVENTS (1-ON-1)

- onlineUsers
- receiveMessage
- userTyping
- messageStatusUpdate
- messageDeleted
- conversationCleared

📤 EMITTED EVENTS (GROUP)

- joinGroup / leaveGroup
- createGroup / deleteGroup
- sendGroupMessage
- getUserGroups
- editGroupName
- addGroupMembers
- removeGroupMember
- groupTyping

📥 LISTENED EVENTS (GROUP)

- groupCreated / groupDeleted
- receiveGroupMessage
- groupMessageRead
- groupMessageDeleted
- groupTyping
- groupMembersAdded
- groupMemberRemoved
- groupNameEdited

6 Storage Strategy

The app uses a **3-tier local storage strategy** — no on-device SQL/NoSQL database. All persistent data lives on the backend; local storage is purely for caching and session management.

Tier 1 — SecureStorageService (Encrypted)

Package: `flutter_secure_storage: ^9.2.4` — stores sensitive data in the OS keychain/keystore.

KEY	DATA
auth_token	JWT Bearer token
employee_id	Logged-in employee ID
username, email, role	User identity fields
app_token	FCM push notification token
mobile, alter_mobile, gender, address, profile_image	Profile fields
nda_submitted, onboarding_completed	Onboarding flags

Tier 2 — LocalStorageService (JSON Cache)

Package: `shared_preferences: ^2.5.3` — stores non-sensitive cached data as JSON strings.

KEY	DATA	TTL
employee_list	Full employee list (JSON array)	1 hour
chat_messages_{conversationId}	Messages per conversation	Persistent
group_chats	Group chat list	Persistent
group_messages_{groupId}	Messages per group	Persistent
meet_list	Meeting list	Timestamped
inbox_mails, sent_mails, trash_mails	Mail cache	Timestamped
leave_history, available_leave	Leave data	Persistent
attendance_data	Attendance records	Persistent
recent_chats	Ordered list of recent chat user IDs	Persistent
is_logged_in	Boolean login flag	Session

Tier 3 — Remote Backend Database

> The backend (AWS + Render) manages the actual persistent database. The app treats it as the source of truth and caches locally for offline access and performance optimization.

7 Push Notifications

A dual notification system handles both remote (FCM) and local notifications:

FIREBASE CLOUD MESSAGING

- Package: firebase_messaging ^16.0.3
- Handles background + terminated state
- FCM token synced to backend on login
- Token auto-refreshes on change
- Stored in SecureStorage

LOCAL NOTIFICATIONS

- Package: flutter_local_notifications ^19.5.0
- Shows in-app foreground notifications
- Consolidated chat badge (e.g. "3 unread")
- Tap navigates to correct chat screen
- Supports Android + iOS

NOTIFICATION TYPE	NAVIGATION TARGET
Message (1-on-1 chat)	/chat/:otherUserId
GroupMessage (group chat)	/chat/group/:groupId

8 Navigation System

Package: `go_router: ^16.2.4` — declarative, URL-based routing with shell routes and auth guards.

⚙️ ROUTER FEATURES

- ShellRoute for persistent bottom nav
- Auth guard (authGuard function)
- Fade + Slide transitions (300ms)
- rootNavigatorKey for global navigation
- Extra params for passing objects

🏠 SHELL ROUTES (BOTTOM NAV)

- /home → HomeScreen
- /chat → ChatListScreen
- /meet-view → MeetScreen
- /mail → MailScreen
- /mail_detail → MailDetailsScreen

All Application Routes

ROUTE	SCREEN	TYPE
/splash	SplashScreen	Public
/login	LoginScreen	Public
/forgot-password	ForgotPasswordScreen	Public
/verify-otp	VerifyOtpScreen	Public
/change-password	ChangePasswordScreen	Public
/nda	NdaScreen	Onboarding
/onboarding	OnboardingFormScreen	Onboarding
/home	HomeScreen	Shell
/chat	ChatListScreen	Shell
/meet-view	MeetScreen	Shell
/mail	MailScreen	Shell
/chat/:otherUserId	ChatScreen (1-on-1)	Protected
/chat/group/:groupId	GroupChatScreen	Protected
/chat/group/:groupId/details	GroupDetailsScreen	Protected
/scedule-meet	ScheduleMeet	Protected
/compose-mail	ComposeMailScreen	Protected

/permissions	PermissionsScreen	Protected
/face-verification	FaceVerification	Protected
/user-profile	ProfileScreen	Protected
/leave-request	LeaveRequestScreen	Protected
/admin-leave-approval	AdminLeaveApprovalScreen	Admin
/holiday-calendar	HolidayCalendarScreen	Protected
/tasks	TasksScreen	Protected
/social-hub	SocialHubScreen	Protected
/resignation	ResignationScreen	Protected
/complaint	MyComplaintsScreen	Protected

9 Firebase Integration

PACKAGE	VERSION	PURPOSE
firebase_core	^4.2.0	Firebase initialization
firebase_messaging	^16.0.3	Push notifications (FCM)
firebase_crashlytics	^5.0.2	Crash reporting & monitoring

> Configured via `firebase_options.dart` (auto-generated by FlutterFire CLI). `firebase.json` present at project root.

10 Theme & UI

FEATURE	DETAIL
Theme Mode	Light mode only (ThemeMode.light)
Material Design	uses-material-design: true
Font Scaling	Clamped 0.8x – 1.2x to prevent layout breaks
Splash Screen	flutter_native_splash (white bg + logo, fullscreen)
App Icon	flutter_launcher_icons (adaptive icon for Android)
Charts	syncfusion_flutter_charts ^31.2.4

Shimmer Loading	shimmer ^3.0.0
Image Caching	cached_network_image ^3.4.1
Photo Viewer	photo_view ^0.15.0
Video Player	video_player ^2.10.0
OTP Input	pinput ^5.0.2
Loading Animations	loading_animation_widget ^1.3.0

11 Authentication Flow

```
── LOGIN FLOW ──────────────────────────────────────
1. User enters employeeId + password
2. POST /employee/login
3. JWT token → SecureStorageService (auth_token)
4. FCM token synced → backend (employee/profile/:id)
5. Socket joined (userId + username)
6. Navigate based on flags:
   ndaSubmitted = false    → /nda
   onboardingCompleted = false → /onboarding
   else                    → /home

── FORGOT PASSWORD FLOW ────────────────────────────
1. GET  /employee/getOTP/:employeeId
2. POST /employee/verifyOtp  { employeeId, otp }
3. POST /employee/changePassword { employeeId, password }

── SESSION RESTORE ─────────────────────────────────
1. App start → LocalStorageService.isLoggedIn()
2. If true → SecureStorageService.getUserData()
3. AuthState.authenticated → navigate to /home

── LOGOUT FLOW ─────────────────────────────────────
1. SocketManager.leave() → disconnect socket
2. SecureStorageService.clearUserData()
3. LocalStorageService.setLoginState(false)
4. Navigate to /login
```

12 Feature Modules

FEATURE	SCREENS	KEY CAPABILITIES
Auth	Login, ForgotPassword, VerifyOTP, ChangePassword, NDA, Onboarding, Profile	JWT auth, OTP flow, profile update, face verification
Chat	ChatList, ChatScreen, GroupChatScreen, GroupDetails	Real-time 1-on-1 & group chat, file sharing, read receipts, typing indicators, delete messages
Home	HomeScreen, ProfileScreen	Dashboard, clock-in/out with face verification, attendance tracking
Mail	MailScreen, MailDetail, ComposeMail	Internal email system with inbox/sent/trash
Meet	MeetScreen, ScheduleMeet	Meeting list and scheduling

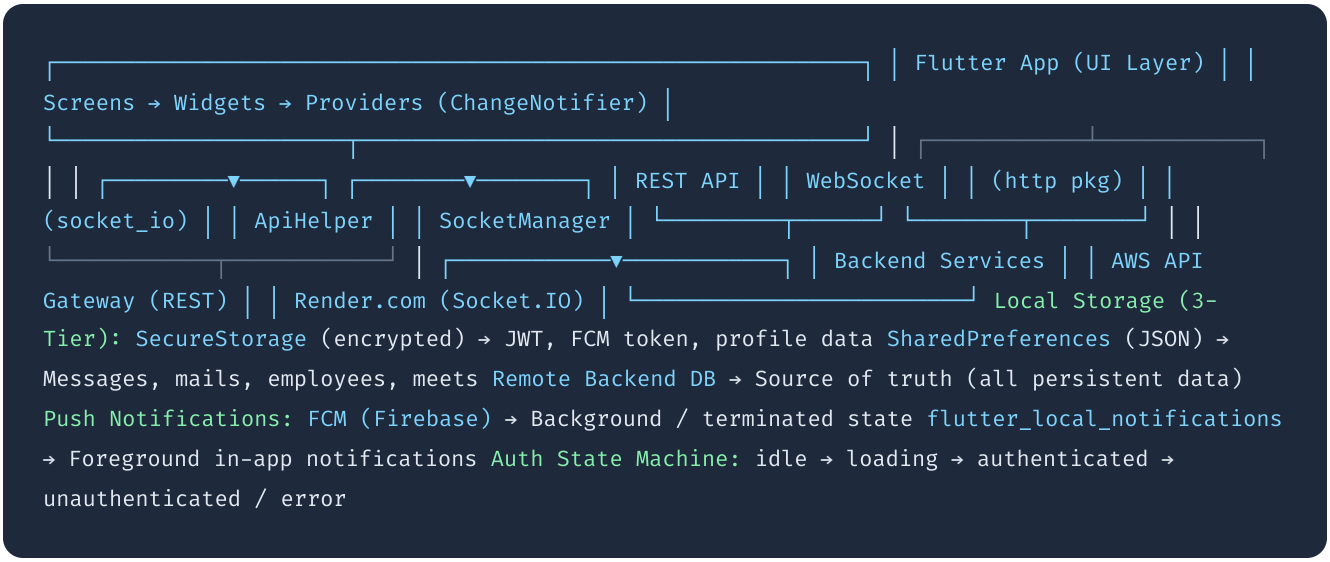
Leave	LeaveRequest, AdminLeaveApproval	Leave requests, history, admin approval/rejection
Holiday	HolidayCalendar	Company holiday calendar
Tasks	TasksScreen	Task management and tracking
Social	SocialHubScreen	Social feed / company posts
Complaint	MyComplaintsScreen	Submit and track complaints
HR Services	ResignationScreen	Submit resignation
Settings	PermissionsScreen	App permission management

13 Key Packages Summary

CATEGORY	PACKAGE	VERSION
State Management	provider	^6.1.5+1
Navigation	go_router	^16.2.4
HTTP Client	http	^1.5.0
HTTP Client (alt)	dio	^5.9.0
WebSocket	socket_io_client	^3.1.2
Secure Storage	flutter_secure_storage	^9.2.4
Local Cache	shared_preferences	^2.5.3
Firebase Core	firebase_core	^4.2.0
Push Notifications	firebase_messaging	^16.0.3
Crash Reporting	firebase_crashlytics	^5.0.2
Local Notifications	flutter_local_notifications	^19.5.0
File Picker	file_picker	^10.3.3
Image Picker	image_picker	^1.2.0
File Opener	open_file	^3.5.10
Path Provider	path_provider	^2.1.5
Video Player	video_player	^2.10.0
Photo Viewer	photo_view	^0.15.0

Image Cache	cached_network_image	^3.4.1
Charts	syncfusion_flutter_charts	^31.2.4
Shimmer	shimmer	^3.0.0
OTP Input	pinput	^5.0.2
Loading Animations	loading_animation_widget	^1.3.0
UUID Generator	uuid	^4.5.1
Internationalization	intl	^0.20.2
Crypto	crypto	^3.0.6
MIME Types	mime	^2.0.0
Logger	logger	^2.6.2
Environment	flutter_dotenv	^6.0.0
Permissions	permission_handler	^12.0.1
URL Launcher	url_launcher	^6.3.2
Audio Player	audioplayers	^6.5.1
Splash Screen	flutter_native_splash	^2.4.7
App Icon	flutter_launcher_icons	^0.14.4

14 Architecture Diagram



> **Note:** The app uses `keys.env` (flutter_dotenv) for environment configuration. Ensure this file is present and not committed to version control. The socket server on Render.com may have cold-start delays (~30s) after inactivity.